Text Splitting — Chunks, Chunk Size, Chunk Overlap & Separators

1. Quick Revision — LangChain Document

A LangChain Document mainly contains:

Document

|

|--- page_content

|

|--- metadata

Example:

```
from langchain_core.documents import Document


document = Document(
    page_content="""
    Employees must report by 9 AM.
    Employees are eligible for casual leave.
    Employees must follow company policies.
    """,
```

```
    metadata={
        "source": "company_policy.txt"
    }
)
```

Access content:

```
print(
    document.page_content
)
```

Access metadata:

```
print(
    document.metadata
)
```

2. Why Do We Need Text Splitting?

Suppose we load a very large PDF.

500-PAGE PDF

↓

DOCUMENT LOADER

↓

LANGCHAIN DOCUMENT

↓

VERY LARGE PAGE CONTENT

Now imagine sending the entire content directly to an LLM.

This can create several problems.

3. Problems with Very Large Documents

Problem 1 — Context Window Limitations

An LLM cannot always accept unlimited text.

Very Large Content

↓

LLM Context Limit

↓

Content May Not Fit

Problem 2 — Unnecessary Information

A user may ask only about:

Leave Policy

But if we send the complete document:

Attendance

Leave

Benefits

Rules

Policies

Other Information

much of the content may be unnecessary.

Problem 3 — More Token Usage and Cost

More text means:

More Input

↓

More Tokens

↓

Potentially More Cost

Problem 4 — Slower Processing

A larger amount of content generally requires more processing.

More Text

↓

More Processing

↓

Potentially Slower Response

Problem 5 — Difficult Information Retrieval

If the whole document remains as one large unit:

One Huge Document

↓

Harder to Select

Small Relevant Portion

Smaller chunks make it easier to search for relevant pieces.

4. Module 9 vs Module 10

Module 9

SOURCE FILE

↓

DOCUMENT LOADER

↓

DOCUMENT OBJECT

Module 9 teaches us:

How to load data.

Module 10

DOCUMENT OBJECT

↓

TEXT SPLITTER

↓

SMALLER CHUNKS

Module 10 teaches us:

How to divide the loaded data.

5. Main Requirement

After loading a large document:

How can we divide its content into smaller useful pieces so that an AI application can process and search the information efficiently?

That process is:

TEXT SPLITTING

6. Without Text Splitting vs With Text Splitting

Without Text Splitting	With Text Splitting
Content may remain one large unit	Content is divided into multiple pieces
Difficult to select a small relevant portion	Smaller pieces can be searched individually
Sending everything may use more tokens	Selected chunks can reduce unnecessary input
Less control over content size	Chunk size can be configured

7. Four Important Terms

Term	Meaning
Chunk	A piece of text
Chunk Size	How large a chunk should be
Chunk Overlap	Repeated content between nearby chunks
Separator	Where text can be divided

8. What Is a Chunk?

Definition

A chunk is a smaller piece of text obtained by dividing a larger text or document.

Example:

Original Text:

ABCDEFGHIJKL

Total:

12 Characters

Suppose we divide into pieces of 4 characters.

Chunk 1: ABCD

Chunk 2: EFGH

Chunk 3: IJKL

So:

Large Text

↓

Split

↓

Smaller Pieces

↓

Chunks

9. Easy Memory

CHUNK

↓

Small Piece of Text

10. What Is Chunk Size?

Definition

Chunk size means the target maximum length of each chunk, measured using a chosen length function.

Suppose:

chunk_size = 5

Text:

ABCDEFGHIJKL

With simple fixed-size splitting:

Chunk 1: ABCDE

Chunk 2: FGHIJ

Chunk 3: KL

11. Important Observation

The chunks contain:

Chunk 1 → 5 characters

Chunk 2 → 5 characters

Chunk 3 → 2 characters

Student question:

If chunk_size = 5, must every chunk contain exactly 5 characters?

No.

Chunk size generally acts as a:

Target Maximum

not a requirement that every chunk must have exactly that size.

12. Chunk Size Memory Formula

chunk_size

↓

Target Maximum

Chunk Length

13. Characters vs Tokens

Important:

Characters

≠

Tokens

They are different concepts.

In today's simple examples, we are often demonstrating length using characters.

14. What Is Chunk Overlap?

Chunk overlap is one of the most important concepts.

Suppose important information appears near a chunk boundary.

Without overlap:

Chunk 1 Ends

↓

Important Information May Split

↓

Chunk 2 Starts

To reduce this problem, we repeat a small amount of content in neighbouring chunks.

That repeated part is called:

CHUNK OVERLAP

15. Chunk Overlap Example

Original text:

ABCDEFGHIJKL

Configuration:

chunk_size = 5

chunk_overlap = 2

Simple sliding chunks:

Chunk 1: ABCDE

Chunk 2: DEFGH

Chunk 3: GHIJK

Chunk 4: JKL

Observe:

Chunk 1 & 2

Shared = DE

Chunk 2 & 3

Shared = GH

Chunk 3 & 4

Shared = JK

16. Why Repeat Content?

Student question:

Why repeat the same content? Is it not a waste of memory?

Yes, overlap requires some extra space.

But controlled overlap can preserve continuity across chunk boundaries.

17. Chunk Overlap Trade-Off

No Overlap

↓

May Break Related Information

Some Overlap

↓

Improves Continuity

Too Much Overlap

↓

Unnecessary Duplication

So:

Overlap is a trade-off between continuity and duplication.

18. What Is a Separator?

Definition

A separator is a character or sequence of characters used to identify possible places for splitting text.

Examples:

Separator	Meaning
"\n\n"	Paragraph boundary
"\n"	Newline
" "	Space
"."	Full stop, when explicitly selected
""	Empty separator, character-level fallback in suitable splitters

19. Separator Memory Formula

Separator

↓

WHERE can text be split?

20. Three Core Questions

Whenever we split text, think of these three questions:

Separator

↓

WHERE to split?

Chunk Size

↓

HOW MUCH text to keep?

Chunk Overlap

↓

HOW MUCH text to repeat?

21. Program 1 — Fixed-Size Chunking Using Python

Objective

Split text into fixed-size chunks without overlap.

```
# ========================================
# ORIGINAL TEXT
# ========================================


text = "ABCDEFGHIJKL"
```

```
# ========================================
# CONFIGURE CHUNK SIZE
# ========================================

chunk_size = 5



# ========================================
# CREATE CHUNKS
# ========================================

chunks = []



for i in range(
    0,
    len(text),
    chunk_size
```

```
):

    chunk = text[
        i:i + chunk_size
    ]

    chunks.append(
        chunk
    )


# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "ORIGINAL TEXT:",
    text
)
```

```
print(
    "ORIGINAL LENGTH:",
    len(text)
)


print(
    "TOTAL CHUNKS:",
    len(chunks)
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"Chunk {index}: {chunk} "
```

```
        f"| Length: {len(chunk)}"
    )
```

22. Program 1 Output

```
ORIGINAL TEXT: ABCDEFGHIJKL
ORIGINAL LENGTH: 12
TOTAL CHUNKS: 3

Chunk 1: ABCDE | Length: 5
Chunk 2: FGHIJ | Length: 5
Chunk 3: KL | Length: 2
```

23. Understanding range()

We use:

```
range(
    0,
    len(text),
    chunk_size
)
```

Here:

len(text) = 12

chunk_size = 5

So:

range(

    0,

    12,

    5

)

produces:

0

5

10

24. Understanding the Starting Index

First iteration:

i = 0

Then:

text[0:5]

gives:

ABCDE

Second iteration:

i = 5

Then:

text[5:10]

gives:

FGHIJ

Third iteration:

i = 10

Then:

text[10:15]

gives:

KL

Python safely stops at the actual string end.

25. Program 1 Flow

ABCDEFGHIJKL

↓

chunk_size = 5

↓

Indexes:

0, 5, 10

↓

ABCDE

FGHIJ

KL

26. Program 2 — Text Splitting with Chunk Overlap

Objective

Create fixed-size chunks with repeated content between neighbouring chunks.

```
# ========================================

# ORIGINAL TEXT

# ========================================


text = "ABCDEFGHIJKL"



# ========================================

# CONFIGURATION

# ========================================
```

```
chunk_size = 5

chunk_overlap = 2



# ========================================
# VALIDATE CONFIGURATION
# ========================================

if not 0 <= chunk_overlap < chunk_size:

    raise ValueError(
        "Overlap must be non-negative "
        "and smaller than chunk size"
    )



# ========================================
# CALCULATE STEP SIZE
```

```
# ========================================

step = (
    chunk_size
    - chunk_overlap
)



# ========================================
# CREATE CHUNKS
# ========================================

chunks = []



for i in range(
    0,
    len(text),
    step
):
```

```
    chunk = text[
        i:i + chunk_size
    ]

    chunks.append(
        chunk
    )


# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "CHUNK SIZE:",
    chunk_size
)
```

```
print(
    "CHUNK OVERLAP:",
    chunk_overlap
)


print(
    "STEP SIZE:",
    step
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"Chunk {index}: {chunk}"
    )
```

27. Program 2 Output

CHUNK SIZE: 5

CHUNK OVERLAP: 2

STEP SIZE: 3

Chunk 1: ABCDE

Chunk 2: DEFGH

Chunk 3: GHIJK

Chunk 4: JKL

28. Understanding Step Size

Important formula:

STEP SIZE

=

CHUNK SIZE - CHUNK OVERLAP

Example:

chunk_size = 5

chunk_overlap = 2

Therefore:

step = 5 - 2

= 3

29. Why Step Size Is 3

Chunk 1 starts at:

Index 0

Chunk 2 starts at:

Index 3

Chunk 3 starts at:

Index 6

Chunk 4 starts at:

Index 9

So:

0 → ABCDE

3 → DEFGH

6 → GHIJK

9 → JKL

30. Overlap Visualisation

Chunk 1: A B C D E

| |

Chunk 2: D E F G H

| |

Chunk 3: G H I J K

| |

Chunk 4: J K L

The repeated text creates continuity.

31. Why Validate Chunk Overlap?

We check:

if not 0 <= chunk_overlap < chunk_size:

This requires:

chunk_overlap >= 0

and:

chunk_overlap < chunk_size

Example of valid configuration:

chunk_size = 5

chunk_overlap = 2

Invalid example:

chunk_size = 5

chunk_overlap = 5

because the overlap must be smaller than the chunk size in this program.

32. Program 3 — Understanding Separators Using Python

Objective

Split text based on paragraph boundaries.

```
# ========================================

# ORIGINAL TEXT

# ========================================


text = (
    "Python is easy.\n\n"
    "LangChain helps create AI applications.\n\n"
    "RAG uses documents."
)
```

```
# ========================================
# DEFINE SEPARATOR
# ========================================

separator = "\n\n"



# ========================================
# SPLIT TEXT
# ========================================

chunks = text.split(
    separator
)



# ========================================
# DISPLAY RESULTS
# ========================================
```

```
print(
    "TOTAL CHUNKS:",
    len(chunks)
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"\nChunk {index}:"
    )

    print(
        chunk
    )
```

33. Understanding "\n\n"

\n

means:

New Line

Therefore:

\n\n

represents:

Two Newlines

which commonly indicates a paragraph break.

34. Program 3 Output Concept

Original:

Python is easy.

LangChain helps create AI applications.

RAG uses documents.

After:

text.split(

"\n\n"

)

we get:

Chunk 1:

Python is easy.

Chunk 2:

LangChain helps create AI applications.

Chunk 3:

RAG uses documents.

35. What Does split() Return?

chunks = text.split(

  separator

)

returns:

list[str]

Conceptually:

[

  "Python is easy.",

```
    "LangChain helps create AI applications.",
    "RAG uses documents."
]
```

36. Python Splitting vs LangChain Splitting

So far, we manually used:

split()

and slicing.

LangChain provides a dedicated package:

langchain-text-splitters

for text splitting.

37. Important LangChain Text Splitter Classes

1. CharacterTextSplitter

Splits text based on

a specified separator

and combines pieces

into chunks

2. RecursiveCharacterTextSplitter

Tries different separators

in sequence

to create suitable chunks

38. Important Methods

Method	Input	Return Type
split_text()	String	list[str]
split_documents()	List of Document objects	list[Document]

39. Easy Memory

split_text()

↓

String In

Strings Out

split_documents()

↓

Documents In

Documents Out

40. Program 4 — First LangChain Text Splitting Program

Objective

Use RecursiveCharacterTextSplitter to split a string.

```
from langchain_text_splitters import (
   RecursiveCharacterTextSplitter
)



# ========================================
# ORIGINAL TEXT
# ========================================

text = "ABCDEFGHIJKL"



# ========================================
# CREATE TEXT SPLITTER
# ========================================

splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=5,
    chunk_overlap=2,
    separators=[""]
)


# ========================================
# SPLIT TEXT
# ========================================

chunks = splitter.split_text(
    text
)


# ========================================
# DISPLAY RESULTS
# ========================================

print(
```

```
    "RETURN TYPE:",
    type(chunks)
)


print(
    "TOTAL CHUNKS:",
    len(chunks)
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"Chunk {index}: {chunk}"
    )
```

41. Program 4 Output

RETURN TYPE: <class 'list'>

TOTAL CHUNKS: 4

Chunk 1: ABCDE

Chunk 2: DEFGH

Chunk 3: GHIJK

Chunk 4: JKL

42. Understanding separators=[""]

We use:

```
separators=[
    ""
]
```

Here:

""

is an empty separator.

In this example, it allows character-level fallback splitting.

Important idea:

First the splitter decides possible split boundaries, then it forms chunks based on the configured size and overlap.

43. Very Important Formula

Separator decides WHERE to split.

Chunk size decides HOW MUCH text to keep in each chunk.

Chunk overlap decides HOW MUCH text to repeat.

44. Splitting vs Chunk Creation

Another important concept:

Splitting decides the boundaries. Merging creates the final chunks. chunk_size controls their maximum size.

So:

Separator

↓

Possible Boundaries

↓

Split Pieces

↓

Merge Pieces

↓

Final Chunks

45. Why Is Splitting Different from Chunking?

Students may think:

"If we split by a separator, chunks are already created. Why is chunk size needed?"

Important:

Splitting

↓

Identifies Smaller Pieces

Then:

Chunk Formation

↓

May Combine Pieces

Based on chunk_size

So:

Separator

↓

Possible Split Points

while:

chunk_size

↓

Controls Final Chunk Size

46. Simple Example

Suppose text contains words:

Python is easy to learn

If separator is:

" "

splitting may first identify:

Python

is

easy

to

learn

But final chunks may combine several pieces depending on:

chunk_size

So:

Separator does not always mean every separated piece becomes a final chunk.

47. Program 5 — Splitting LangChain Document Objects

Objective

Split a LangChain Document into smaller LangChain Document chunks.

```
from langchain_core.documents import (
    Document
)

from langchain_text_splitters import (
    RecursiveCharacterTextSplitter
)



# ========================================
# CREATE DOCUMENT
# ========================================

document = Document(
    page_content=
        "ABCDEFGHIJKL",
```

```
    metadata={
        "source":
            "sample.txt"
    }
)



# ========================================
# CREATE TEXT SPLITTER
# ========================================

splitter = RecursiveCharacterTextSplitter(
    chunk_size=5,
    chunk_overlap=2,
    separators=[""]
)



# ========================================
```

```
# SPLIT DOCUMENT
# ========================================

chunks = splitter.split_documents(
    [
        document
    ]
)



# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "TOTAL DOCUMENT CHUNKS:",
    len(chunks)
)
```

```
for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"\nCHUNK {index}"
    )


    print(
        "CONTENT:",
        chunk.page_content
    )


    print(
        "METADATA:",
        chunk.metadata
    )
```

```
    print(
        "TYPE:",
        type(chunk).__name__
    )
```

48. Program 5 Output

```
TOTAL DOCUMENT CHUNKS: 4
Chunk 1:
CONTENT: ABCDE
METADATA: {'source': 'sample.txt'}
TYPE: Document
Chunk 2:
CONTENT: DEFGH
METADATA: {'source': 'sample.txt'}
TYPE: Document
Chunk 3:
CONTENT: GHIJK
METADATA: {'source': 'sample.txt'}
```

TYPE: Document

Chunk 4:

CONTENT: JKL

METADATA: {'source': 'sample.txt'}

TYPE: Document

49. Important Observation — Metadata Is Preserved

Original Document metadata:

```
{
   "source": "sample.txt"
}
```

After splitting:

Chunk 1

↓

{'source': 'sample.txt'}

Chunk 2

↓

{'source': 'sample.txt'}

Chunk 3

↓

{'source': 'sample.txt'}

Chunk 4

↓

{'source': 'sample.txt'}

This is very useful because every chunk still knows where it came from.

50. split_text() vs split_documents()

split_text()

Input:

"ABCDEFGHIJKL"

Output:

[

"ABCDE",

"DEFGH",

"GHIJK",

"JKL"

]

Return type:

list[str]

split_documents()

Input:

[

Document(...)

]

Output:

[

Document(...),

Document(...),

Document(...),

Document(...)

]

Return type:

list[Document]

51. Most Important Difference

split_text()

↓

TEXT → TEXT CHUNKS

split_documents()

↓

DOCUMENTS → DOCUMENT CHUNKS

52. Why split_documents() Is Useful

Suppose a document contains:

page_content

+

metadata

If we use split_documents():

Original Document

↓

Split

↓

Document Chunk 1

Document Chunk 2

Document Chunk 3

Metadata remains associated with the chunks.

This is important later for:

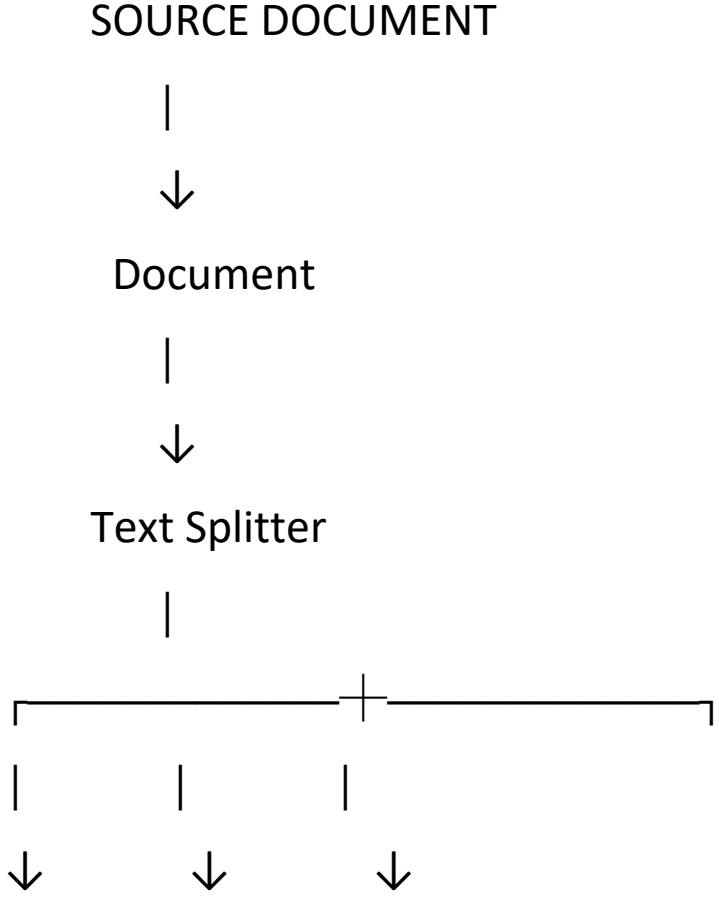
Retrieval

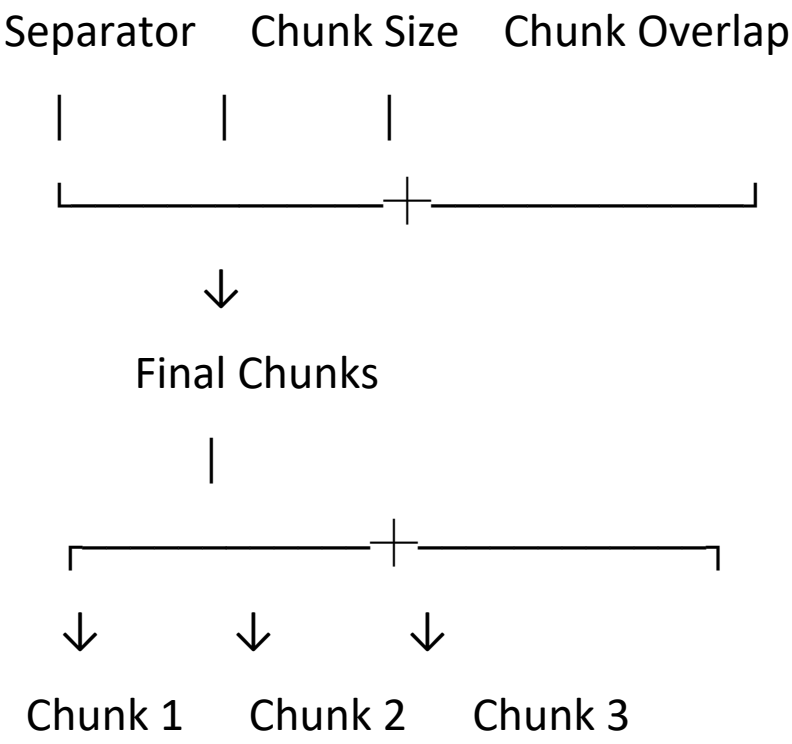
Source Tracking

RAG

Filtering

53. Complete Text Splitting Architecture





54. Important Concept — Separator

Separator

↓

Possible Place to Split

Examples:

Paragraph

Newline

Space

Character

55. Important Concept — Chunk Size

Chunk Size

↓

Target Maximum Size

of Final Chunk

It does not mean every chunk must be exactly equal in size.

56. Important Concept — Chunk Overlap

Chunk Overlap

↓

Repeated Content

Between Neighbouring Chunks

Purpose:

Preserve Context

Across Boundaries

57. Common Mistake — Thinking Every Chunk Must Equal chunk_size

Wrong:

chunk_size = 5

↓

Every Chunk Must Be Exactly 5

Correct:

chunk_size = 5

↓

Target Maximum

Final chunk may be shorter.

58. Common Mistake — Thinking Chunk Overlap Adds New Information

No.

Chunk overlap:

Repeats Existing Information

It does not create new information.

Example:

ABCDE

DEFGH

DE is repeated.

59. Common Mistake — Thinking More Overlap Is Always Better

No.

Very High Overlap

↓

Too Much Duplicate Content

↓

More Storage / Processing

Use controlled overlap.

60. Common Mistake — Thinking Separator Is the Final Chunk Size

No.

Separator answers:

WHERE can we split?

Chunk size answers:

HOW LARGE should chunks become?

61. Common Mistake — Thinking split() and LangChain Splitters Are the Same

Python:

```
text.split(
   separator
)
```

performs direct splitting based on a separator.

LangChain text splitters may consider:

Separators

Chunk Size

Chunk Overlap

Chunk Construction

So the LangChain splitter provides more chunk-management logic.

62. Common Mistake — Thinking separators=[""] Means No Splitting

No.

In the supplied example:

```
separators=[""]
```

is used to enable character-level fallback splitting.

So:

Empty Separator

↓

Character-Level Splitting

in this example

63. Common Mistake — Confusing split_text() and split_documents()

Wrong:

```
split_text(
   [
      Document(...)
   ]
)
```

The supplied notes define:

split_text()

↓

String Input

and:

split_documents()

↓

List of Document Objects

64. Program Summary

Program 1

Python Fixed-Size Splitting

↓

No Overlap

Program 2

Python Splitting

↓

With Overlap

Program 3

Python split()

↓

Separator-Based Splitting

Program 4

RecursiveCharacterTextSplitter

↓

split_text()

Program 5

LangChain Document

↓

split_documents()

↓

Document Chunks

65. Today's Master Formula

LARGE DOCUMENT

↓

TEXT SPLITTER

↓

SEPARATOR

↓

CHUNK SIZE

↓

CHUNK OVERLAP

↓

SMALLER CHUNKS

66. RAG-Oriented Flow

SOURCE FILE

↓

DOCUMENT LOADER

↓

DOCUMENT

↓

TEXT SPLITTER

↓

CHUNKS

↓

Later Processing / Retrieval

67. Five Important Revision Points

1. Chunk

↓

Small Piece of Text

2. Chunk Size

↓

Target Maximum Chunk Size

3. Chunk Overlap

↓

Repeated Content

Between Neighbouring Chunks

4. Separator

↓

Possible Splitting Boundary

5. split_documents()

↓

Creates Document Chunks

and Preserves Metadata

68. Final Memory Formula

SEPARATOR

↓

WHERE?

CHUNK SIZE

↓

HOW MUCH?

CHUNK OVERLAP

↓

HOW MUCH TO REPEAT?

69. One-Line Conclusion

Text splitting divides large Document content into smaller chunks; separators decide where splitting can happen, chunk size controls how large chunks should be, and chunk overlap repeats some nearby content to preserve continuity.